02-ContextEngineering 보충 자료

SRE 에이전트로 보는 컨텍스트 엔지니어링

LangGraph Supervisor + 전문 에이전트 4개 샘플로 확인하는 기록 · 선택 · 압축 · 분리 · 도구 설계
Building Advanced Agentic Systems on AWS

입력 형식
`json.dumps(indent=4)`

기록
`create_event` · 전략 3개

선택
`retrieve_memories`

압축
500자 · 9,000자 절단

분리
`agent_results`

이 자료의 구성

A

교안 ↔ 코드 매핑
교안 슬라이드를 진행하며 화면에 띄울 코드 위치 13곳과 5대 전략 진단

B

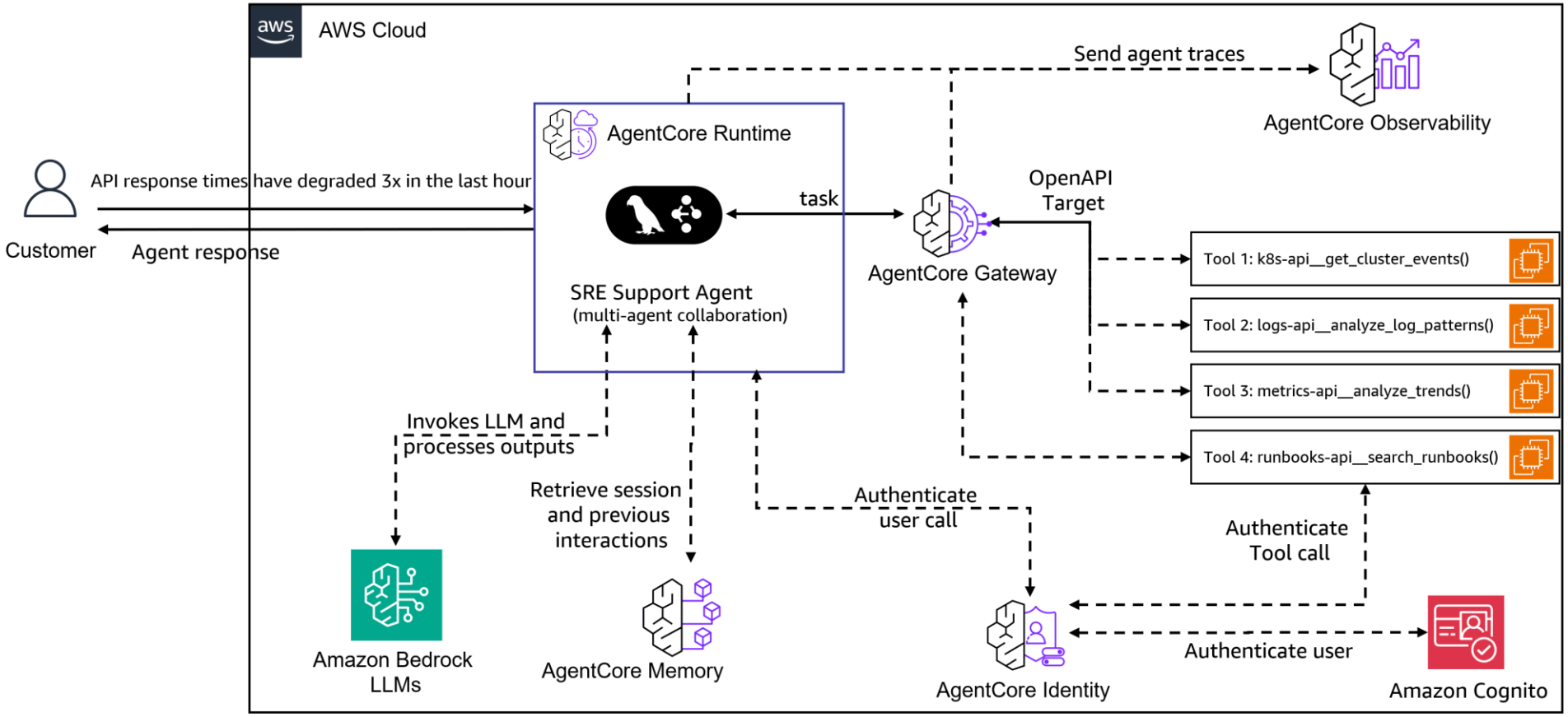
추가적인 기술 내용
기억의 이중 주입, 전달되지 않는 계획, 실행 모드별 세션 기록, 컨텍스트 크기, 캐시

C

코드에서 찾은 함정 · 실습 과제
분리 · 선택 · 도구 · 지침 충돌 문제 6가지와 개선 코드

샘플 아키텍처: 한 프로세스, 공유 상태 하나

교안 #13

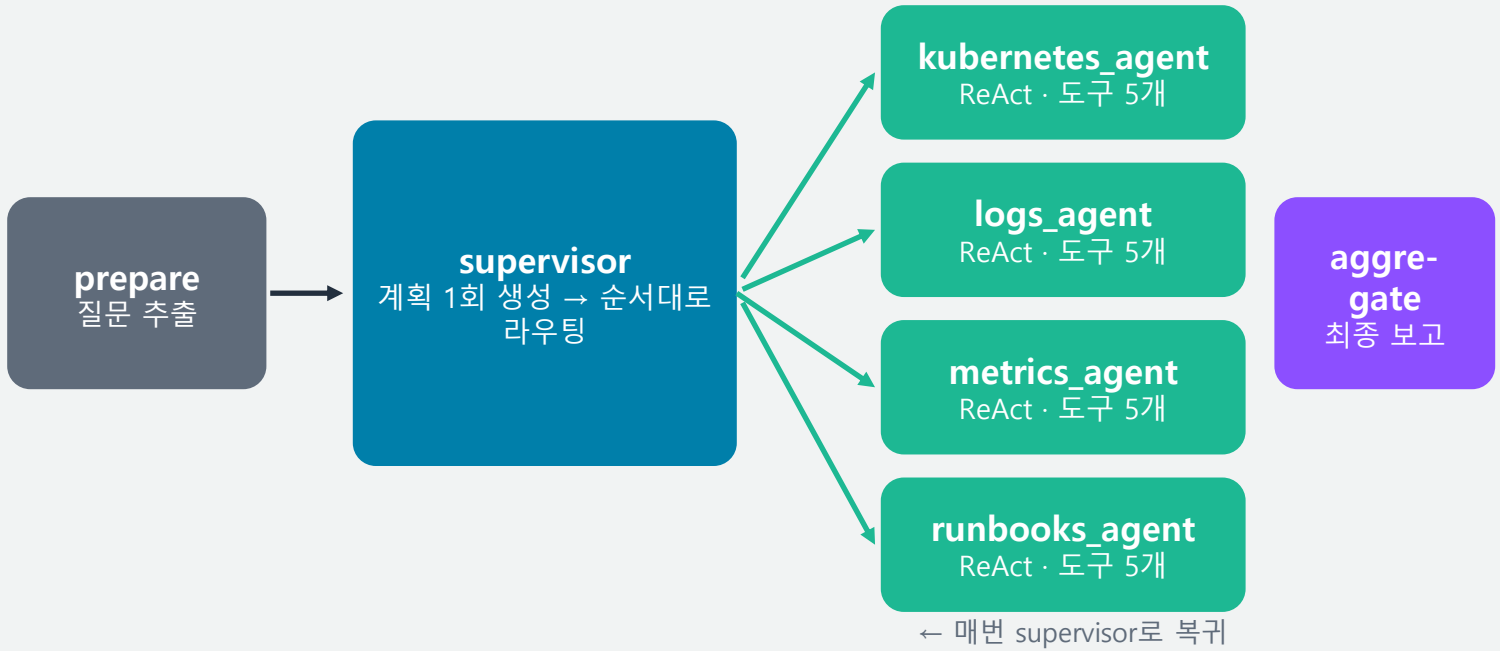


샘플 아키텍처: 한 프로세스, 공유 상태 하나

교안 #13

LangGraph StateGraph(AgentState) · graph_builder.py

공유 AgentState `messages · agent_results · metadata · memory_context`



AgentCore Gateway
OpenAPI 4개 → MCP 도구 20개 + 시맨틱 검색

AgentCore Memory
UserPreference · Semantic · Summary 전략 3개

AgentCore Runtime
agent_runtime.py · POST /invocations

A2A 샘플과의 차이 에이전트 사이에 네트워크 경계가 없다 — 모든 노드가 같은 AgentState를 읽고 쓴다. 분리는 코드로 직접 만들어야 한다

컨텍스트 지도: 세 종류의 LLM 호출이 보내는 것

교안 #5–6

	Supervisor · 계획 planning_agent (ReAct)	전문 에이전트 × 4 create_react_agent	Aggregate · 요약 executive summary LLM
시스템	지침 22KB + 질문 + 메모리 JSON + 계획 지침 supervisor.py:394–399	공통 지침 5.4KB + 에이전트별 지침 prompt_loader.py:97	요약 지침 2.9KB output_formatter.py:186
도구	메모리 도구 4개 (retrieve · save × 3) supervisor.py:235	도메인 도구 5개 (검색 도구 탈락) agent_config.yaml	없음 –
기록 · 메모리	매번 새 메시지 2개 + 도구 결과 supervisor.py:405	공유 state.messages 전체 (이전 에이전트 도구 결과 포함) agent_nodes.py:220	agent_results (최종 텍스트만) + 선호 output_formatter.py:172
이번 입력	"Create an investigation plan for: ..." supervisor.py:408	"As the {name}, help with: {query}" agent_nodes.py:163	질문 + 결과 텍스트 템플릿 executive_summary_user

핵심 기억은 Supervisor만 보고, 도구 결과는 공유 messages를 타고 다음 에이전트로 흐르며, 요약 단계만 결론 텍스트로 분리된다

A

교안 ↔ 코드 매핑

교안을 진행하면서 화면에 띄울 코드 위치와 5대 전략 진단

유한한 리소스 · 최적화 전략: 꼭 짚을 코드 8곳

교안 #5–36

교안	개념	코드 위치	짚을 포인트
#5	컨텍스트 = 메시지 배열	`sre_agent/agent_nodes.py:219-221`	system + state.messages 전체 + 집중 질문
#13	위임 규모 지침	`config/prompts/supervisor_planning_prompt.txt`	'2–3단계, 대부분 1–2 에이전트' 지침 있음
#16	입력 형식	`sre_agent/supervisor.py:354-368`	기억을 json.dumps(indent=2 · 4)로 삽입
#18 · 25	기록	`sre_agent/memory/conversation_manager.py:157`	에이전트마다 USER · ASSISTANT · TOOL 묶음 저장
#20	캐시를 위한 구조화	`sre_agent/supervisor.py:394-399`	정적 22KB + 동적 질문 · 기억 + 정적 지침
#27–29	선택	`sre_agent/memory/hooks.py:46-91`	3종 검색, 점수 필터 없음, 인프라 최대 50개
#30–33	압축	`sre_agent/agent_nodes.py:324`	규칙 기반 절단 (도구 500자, 메시지 9,000자)
#34–36	분리	`sre_agent/output_formatter.py:172`	요약만 결론 텍스트로 분리, 에이전트 사이는 공유

컨텍스트 = 메시지 배열

교안 #5

logs_agent 차례(앞서 metrics_agent 실행)에 모델로 가는 리스트 — agent_nodes.py:219–221

`{"messages": [`

system
[0]

SystemMessage 에이전트 지침 · 공통 5.4KB + 에이전트별

state.messages
공유 · 전체 · 누적

HumanMessage 사용자 질문 (대화형이면 이전 턴부터)

AIMessage metrics_agent의 tool_calls

ToolMessage metrics 도구 결과 원본

AIMessage metrics_agent의 최종 답

… 에이전트가 끝날 때마다 뒤에 누적 (agent_nodes.py:427)

user
[-1] 집중 질문

HumanMessage "As the logs_agent, help with: {query}"

`]}` → `self.agent.astream(...)` 매 ReAct 스텝마다 통째로 재전송

```
# sre_agent/agent_nodes.py
203  system_message = SystemMessage(
         content=self._get_system_prompt())
204  user_message = HumanMessage(content=agent_prompt)
219  async for chunk in self.agent.astream(
220      {"messages": [system_message] + messages
                     + [user_message]}
221  ):
```

+ **앞** 지침은 매 호출 새로 만든 SystemMessage — 상태에 저장되지 않는다

+ **가운데** messages = state["messages"] — 앞선 에이전트의 도구 호출·결과까지 필터 없이 그대로

+ **뒤** agent_prompt로 '이번에 할 일'만 짧게 지정 (agent_nodes.py:163)

핵심 모델이 보는 컨텍스트는 이 리스트 하나뿐이다 — 무엇을 넣고 빼는지가 곧 컨텍스트 엔지니어링이며, 지금은 가운데가 무조건 전부 들어간다

5대 전략 진단: 샘플은 어디까지 구현했나

교안 #15

입력 형식

부분

샘플 구현

메모리에 저장할 때 도구 결과 500자, 대화 9,000자로 절단

빠진 것

기억 레코드 전체(ID · 네임스페이스 등)를 들여쓴 JSON으로 삽입

`supervisor.py:365`
`agent_nodes.py:324`

기록

구현

샘플 구현

대화 묶음과 선호 · 인프라 지식 · 조사 요약을 전략 3개로 기록

빠진 것

저장한 단기 이벤트를 다시 읽는 코드가 없음

`client.py:110-139`
`agent_nodes.py:355`

선택

부분

샘플 구현

계획 시작 시 선호 10 · 인프라 50 · 조사 5개를 검색

빠진 것

점수 필터 없음. 전문 에이전트에게는 전달되지 않음

`hooks.py:46-91`

압축

부분

샘플 구현

조사 요약은 타임라인 200자 · 정규식 발견 5개, 최종 보고는 요약 LLM

빠진 것

대화 관리자 없음. 공유 messages는 계속 증가

`hooks.py:439-494`

분리

부분

샘플 구현

에이전트별 지침 · 도구 5개. 요약 단계는 agent_results만 봄

빠진 것

이전 에이전트의 도구 결과가 다음 에이전트에 그대로 전달

`agent_nodes.py:427`

입력 형식: 내용물보다 큰 포장

교안 #16–17

기억을 프롬프트에 넣을 때, 내용물보다 포장이 더 크다 — supervisor.py:354–368

① **기억 검색 결과**
레코드 리스트 · 최대 10 + 50 + 5건

→

② **`json.dumps(…, indent=2 · 4)`**
dict를 그대로 줄바꿈 + 공백 들여쓰기 문자열로

→

③ **계획 프롬프트에 붙이기**
Supervisor의 SystemMessage 안으로

지금 인프라 기억 1건이 프롬프트에 들어가는 모양 (indent=4)

```
{
    "memoryRecordId": "mem-7f3a9c2e1b4d",
    "content": {
        "text": "web-app-deployment: production ns,
            3 replicas, uses configmap database-config"
    },
    "memoryStrategyId": "infrastructure_knowledge-a1B2c3",
    "namespaces": [
        "/sre/infrastructure/alice/sess-0912"
    ],
    "createdAt": "2026-09-12T08:41:07+00:00",
    "score": 0.4821
}
```

363자

필요한 것 content.text만

```
- web-app-deployment: production ns,
  3 replicas, uses configmap database-config
```

모델에게 쓸모 있는 정보는 이 한 줄이다.

indent=4란? 한 단계 들어갈 때마다 줄 앞에 공백 4칸. 중첩이 깊을수록 공백도 늘어난다

79자

핵심 포장(ID · 메타데이터 · 들여쓰기 공백)도 전부 토큰이다 — 인프라 기억은 최대 50건이 이 모양 그대로 들어간다

기록: 네 갈래 입력, 세 개의 전략

교안 #18, #25–26

① 에이전트 대화 묶음
USER · ASSISTANT · TOOL(500자)
`agent_nodes.py:355`

② 사용자 선호
응답 속 이메일 · #채널 (정규식) `hooks.py:250`

③ 인프라 지식
응답 속 ```json``` 블록 `hooks.py:330`

④ 조사 요약
aggregate 후 규칙 기반 요약 `hooks.py:196`

create_event
actor = user_id

UserPreference
`/sre/users/{actorId}/preferences`

Semantic (인프라 지식)
`/sre/infrastructure/{actorId}/{sessionId}`

Summary (조사 요약)
`/sre/investigations/{actorId}/{sessionId}`

```
# memory/client.py:176 — save_event
messages = [(str(event_data), "ASSISTANT")]
# → "{'timestamp': datetime.datetime(2025, …), …}"
```

입력 형식 관점 구조화 데이터를 str(dict)로 저장해 파이썬 repr이 그대로 추출 LLM에 들어간다. ③은 에이전트가 응답에 JSON 블록을 써야 해서 그 블록이 최종 보고에도 섞인다

기억 전략 3종: memory/strategies.py

교안 #25–29

쓸 때는 Pydantic 구조로 저장하고, AgentCore 전략이 정리한 레코드를 다시 모델로 복원한다

`model_dump()` Pydantic → dict → `save_event()` create_event로 원본 저장 → **`AgentCore 전략`** 비동기로 레코드 추출 → `retrieve_memories()` 네임스페이스 + 시맨틱 검색 → `_retrieve_*()` content.text 파싱 → 모델

	사용자 선호 `UserPreference`	**인프라 지식** `InfrastructureKnowledge`	**조사 요약** `InvestigationSummary`
AgentCore 전략	**UserPreference 전략** `/sre/users/{actorId}/preferences`	**Semantic 전략** `/sre/infrastructure/{actorId}/{sessionId}`	**Summary 전략** `/sre/investigations/{actorId}/{sessionId}`
저장 필드	`preference_type` · `preference_value` · `context`	`service_name` · `knowledge_type` · `knowledge_data` · `confidence`(기본 0.8)	`incident_id` · `query` · `timeline` · `actions_taken` · `resolution_status` · `key_findings`
다시 읽을 때	JSON {preference, categories} → categories로 type 추론	JSON 실패 시 평문 → service_name="general"로 감쌈	XML <summary><topic> → 정규식 추출, key_findings 500자 절단

포인트 저장한 구조가 그대로 돌아오지 않는다 — 전략마다 결과 형식(JSON · 평문 · XML)이 달라 읽는 쪽이 분기 파싱한다.
이 파서는 메모리 도구(tools.py)에서만 쓰이고, 계획 단계 검색(hooks.py)은 레코드 원본을 그대로 프롬프트에 넣는다

선택: 계획 시작 시 세 가지 검색

교안 #27–29

1 사용자 선호

고정 질의

`"user settings communication escalation …"`

top_k 10 · 텍스트만

2 인프라 지식

질의 = 사용자 질문 · 세션 간

`/sre/infrastructure/{user}`

top_k 50 · 레코드 전체

3 과거 조사

질의 = 사용자 질문 · 세션 간

`/sre/investigations/{user}`

top_k 5 · 레코드 전체

→

계획 프롬프트에 삽입

`json.dumps(…, indent=2)`

에이전트별로 묶어 `indent=4`

점수 필터 · 필드 정리 없음

```
# memory/hooks.py:57 — 인프라 지식
all_knowledge = self.memory_client.retrieve_memories(
    memory_type="infrastructure", actor_id=user_id, query=query,
    max_results=SREConstants.memory.max_infrastructure_results)  # 50
knowledge_by_agent = self._organize_memories_by_agent(all_knowledge)
```

전달되지 않는 정리 '에이전트별로 나눠 나중에 전달'하려고 묶었지만, prompt_loader의 memory_context 인자를 아무도 넘기지 않아 전문 에이전트는 기억을 못 본다

교안 #29와 비교 교안은 topK로 개수를, 점수로 품질을 거른다. 샘플은 개수만 정하고(최대 50), 레코드 메타데이터까지 그대로 넣는다

압축: 글자 수 절단, 정규식 요약, 요약 LLM

교안 #30–33

저장할 때 절단

도구 결과 500자로 잘라 TOOL 메시지로 저장

대화 메시지 9,000자 초과 시 [TRUNCATED]

성격 정보 손실을 감수한 길이 제한

`agent_nodes.py:324 · conversation_manager.py:143`

조사 요약 (규칙 기반)

타임라인 에이전트 결과 앞 200자

핵심 발견 'found: ...' 정규식 상위 5개

상태 resolved · escalat 키워드로 판정

`memory/hooks.py:439-510`

최종 보고 (요약 LLM)

입력 agent_results 텍스트 + 선호

출력 executive summary (max_tokens 1000)

주의 뒤에 에이전트 원문을 그대로 덧붙임

`output_formatter.py:88-186`

```
# agent_nodes.py — 교안 #31 대화 관리 적용 예 (LangGraph)
from langchain_core.messages import trim_messages
history = trim_messages(messages, max_tokens=12,
                        token_counter=len,  # 메시지 수 기준
                        strategy="last", start_on="human")
{"messages": [system_message] + history + [user_message]}
```

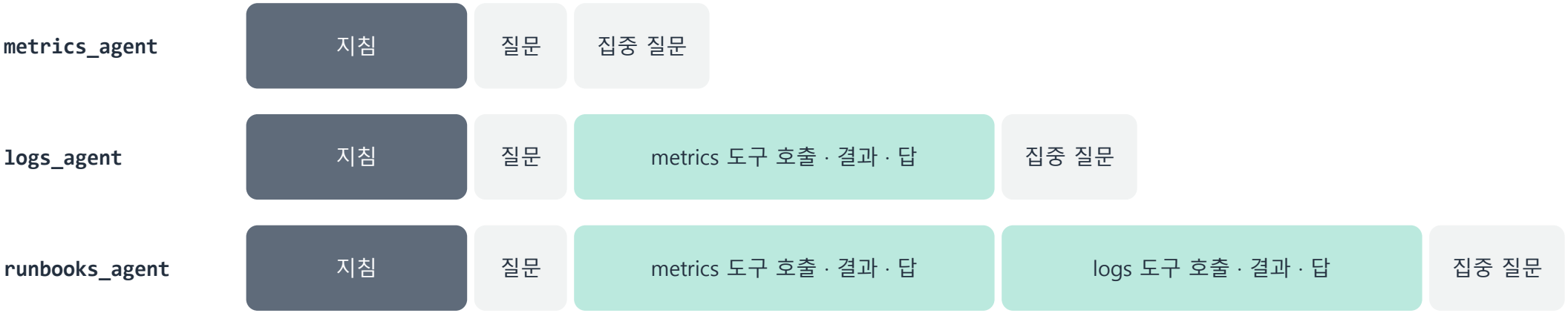
빠진 것: 세션 안 대화 관리 (교안 #31)

모델에 보내는 messages를 줄이는 장치가 없습니다. 압축은 '저장'과 '보고' 쪽에만 있고, 정작 '추론' 입력은 그대로입니다.

분리: 공유 messages가 경계를 지운다

교안 #34–36

계획이 metrics → logs → runbooks일 때 각 에이전트가 받는 입력




```
# agent_nodes.py:219, 427
self.agent.astream({"messages": [system_message] + messages + [user_message]})
...
return {"messages": messages + all_messages,  # 도구 trace까지 추가
        "agent_results": {..., self.name: agent_response}}
```

분리가 있는 곳 aggregate의 요약 LLM은 agent_results(결론 텍스트)만 본다 — 교안 #35의 '요약된 출력만 넘긴다'가 마지막 단계에만 적용됨

빈틈 다음 에이전트에게도 agent_results 같은 결론만 넘기면 분리가 완성된다 — 지금은 다른 도메인의 도구 결과 원본이 그대로 쌓인다

도구 설계: 꼭 짚을 코드 5곳

교안 #39–47

교안	개념	코드 위치	짚을 포인트
#39	도구 응답의 컨텍스트 사용	`backend/servers/logs_server.py:170`	search_logs는 100건 제한, get_error_logs는 무제한
#41–43	정보 집약형 응답	`backend/openapi_specs/*.yaml.template`	REST 응답 원형을 MCP 도구로 그대로 노출
#44–45	Gateway 중앙화	`gateway/main.py:287-322`	OpenAPI 4개 → 도구 20개, 모든 에이전트가 공유
#46	시맨틱 도구 검색	`sre_agent/config/agent_config.yaml:47`	이름 불일치(- vs _)로 검색 도구가 필터에서 탈락
#47	집중형 도구	`sre_agent/agent_nodes.py:37-78`	에이전트별 도구 5개만 바인딩 — 교안 그대로

비교 포인트 도구 '정의'는 잘 나눴지만(에이전트별 5개), 도구 '결과'는 공유 messages를 통해 모든 다음 에이전트에게 퍼집니다.
교안 #47의 집중형 도구와 #34의 분리는 함께 있어야 효과가 납니다.

B

추가적인 기술 내용

교안에는 없지만 샘플의 컨텍스트 흐름을 이해하려면 알아야 할 내부 동작

계획 단계는 같은 기억을 두 번 넣는다

NEW · Planning

경로 ① 훅이 미리 검색해 시스템 메시지에

on_investigation_start() → 선호 · 인프라 · 조사
→ json.dumps 후 planning_prompt에 삽입

supervisor.py:324, 354–399

경로 ② 프롬프트가 도구로 다시 검색하라고 지시

"CRITICAL: … you MUST use the retrieve_memory tool"
→ 선호 · 인프라 · 조사를 도구 결과로 한 번 더

supervisor_planning_prompt.txt <memory_retrieval>



지침 충돌 (교안 #12)

"2-3 steps maximum"

vs

"steps: Array of 3-5 strings"

같은 파일 안에서 단계 수가 다르다. InvestigationPlan 필드 설명도 '3-5'

정리 기억을 넣는 방법은 '훅(결정적)'과 '도구(LLM 판단)' 중 하나로 정해야 한다. 둘 다 쓰면 같은 내용이 두 번 컨텍스트를 쓰고, 도구 호출 왕복만큼 지연도 늘어난다. 훅으로 이미 넣었다면 프롬프트의 <memory_retrieval> 블록을 지운다.

계획은 에이전트에게 전달되지 않는다

NEW · Routing

Supervisor가 만든 계획

```
steps[0]: "Check error rates and latency for payment-service"
steps[1]: "Search error logs around the spike window"
agents_sequence: [metrics_agent, logs_agent]
```

✗

에이전트가 실제로 받는 집중 질문

```
"As the Performance Metrics Agent, help with: {current_query}"
```

단계 설명 없이 원래 질문 전체를 다시 해석

```
# supervisor.py:666-678 — 단계 설명의 쓰임
step_description = plan.steps[next_step]
return {"next": next_agent, "metadata": {
    "routing_reasoning":
        f"Executing plan step {n}: {step_description}"}}
# → 화면 표시 · 로그용. 에이전트 프롬프트에는 없음
```

```
# agent_nodes.py:163 — 개선 예
step = state.get("metadata", {}).get("routing_reasoning", "")
agent_prompt = (f"As the {self.name}, help with: "
                f"{state.get('current_query', '')}\n"
                f"Your assigned step: {step}")
```

결과 에이전트마다 질문 전체를 처음부터 해석하고, 공유 messages의 이전 trace까지 보며 범위를 스스로 정한다 — 교안 #13의 '명확한 역할 분담'이 계획에서 멈춘다

세션 기록은 실행 모드마다 다르다

NEW · Session

실행 모드	턴 사이 messages	이어지는 질문의 맥락	컨텍스트 결과
CLI 대화형	질문과 최종 보고를 계속 누적	전체 대화 (최종 보고 전문 포함)	**턴마다 증가 · /clear 전까지**
AgentCore Runtime	매 호출 [HumanMessage] 하나	장기 메모리 검색으로만	**작지만 직전 대화를 모름**

```
# multi_agent_langgraph.py:853, 1056 — CLI
messages.append(HumanMessage(content=cleaned_query))
...
messages.append(AIMessage(content=final_response))
```

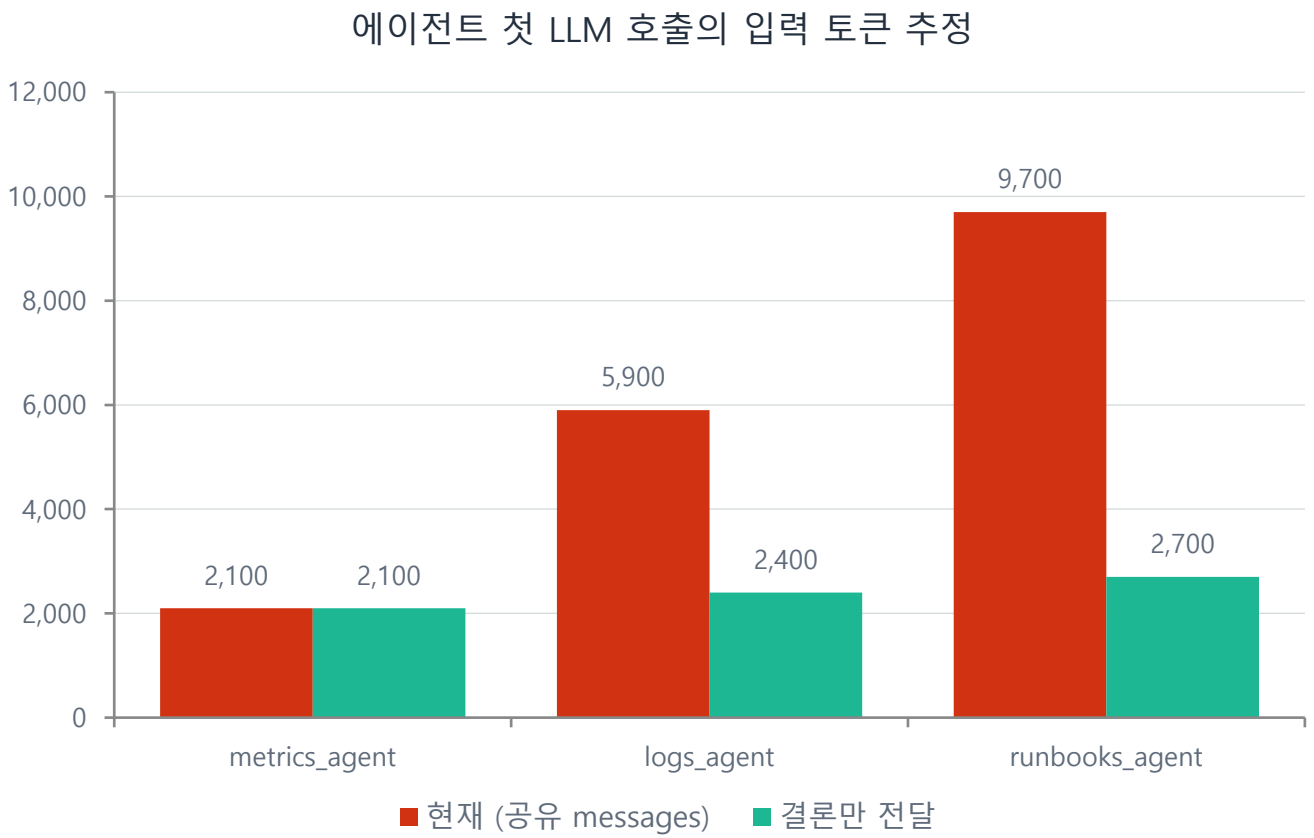
```
# agent_runtime.py:131 — Runtime
initial_state = {
    "messages": [HumanMessage(content=user_prompt)],
    "session_id": session_id, ...}  # checkpointer 없음
```

저장만 하는 단기 기억 에이전트마다 대화 묶음을 create_event로 남기지만, list_events · get_last_k_turns로 다시 읽는 코드가 없다

교안 #31 관점 세션 안: Runtime은 기록 0, CLI는 무제한. 둘 사이의 '최근 N턴 + 요약'이 빠져 있다

에이전트가 이어질수록 입력이 커진다

NEW · Growth



3.6×

세 번째 에이전트의 입력 · 9,700 vs 2,700토큰

ReAct 루프가 곱한다
에이전트 안에서 도구를 한 번 부를 때마다 이 입력 전체가 다시 전송됩니다. 도구 3회면 같은 접두사를 4번 보냅니다.

추정 조건 지침 ≈ 2,050토큰(공통 5.4KB + 에이전트별 ≈ 2.8KB), 질문 ≈ 50, 에이전트당 도구 3회 × 1,000 + 답 800 = 3,800, 결론만 전달 시 에이전트당 300. 4자 ≈ 1토큰

계획 호출은 별도 Supervisor 계획 호출은 고정 지침만 22KB(≈ 5,500토큰)에 기억 JSON(최대 65개 레코드)이 더해진다

캐시 관점: 정적 지침 사이에 끼인 동적 내용

NEW · 교안 #19–24

현재 · planning_prompt (SystemMessage 하나)



정적인 계획 지침이 동적 내용 뒤에 있어 매번 다른 접두사가 된다. 캐시 설정도 없음

개선



정적 지침 ≈ 6,000토큰이 고정 접두사가 되어 매 계획 호출에서 재사용된다

최소 토큰 조건 (교안 #24)

Supervisor 지침만으로 Claude Haiku 4.5의 최소 캐시 길이 4,096을 넘는다 → 이 샘플에서 가장 확실한 캐시 후보

에이전트 지침 (교안 #21)

공통 + 에이전트별 ≈ 2,000토큰은 단독으로는 부족. ReAct 루프가 같은 접두사를 반복하므로 대화 기록까지 포함한 지점이 후보

효과 확인 (교안 #19)

ChatBedrock 응답의 usage_metadata에서 캐시 읽기 · 쓰기 토큰을 비교. 샘플에는 캐시 설정이 없음

도구 설계: 에이전트별 필터와 사라진 검색 도구

교안 #44–47

OpenAPI 명세 4개
k8s · logs · metrics · runbooks

→

AgentCore Gateway
MCP · searchType: SEMANTIC

→

도구 21개
20개 + x_amz_…_search

→

_filter_tools_for_agent
agent_config.yaml 기준

kubernetes	get_pod_status · get_deployment_status · get_cluster_events · get_resource_usage · get_node_status
logs	search_logs · get_error_logs · analyze_log_patterns · get_recent_logs · count_log_events
metrics	get_performance_metrics · get_error_rates · get_resource_metrics · get_availability_metrics · analyze_trends
runbooks	search_runbooks · get_incident_playbook · get_troubleshooting_guide · get_escalation_procedures · …

```
# config/agent_config.yaml:46
global_tools:
  - x-amz-bedrock-agentcore-search   # 하이픈

# Gateway가 내놓는 실제 이름
x_amz_bedrock_agentcore_search       # 밑줄
# → base_tool_name in allowed_tools 실패 → 탈락
```

교안 #47 그대로 도구 20개를 모두 주지 않고 도메인별 5개만 바인딩 → 도구 정의 토큰과 잘못된 도구 선택(혼동)을 함께 줄인다

설정을 믿지 말 것 검색 도구는 어떤 에이전트에도 들어가지 않는다. 도구 5개라 피해는 없지만, 로그의 'has access to N tools'로 확인해야 한다

코드에서 찾은 함정: 토론 · 실습 과제

교안 #12–47

1 에이전트 간 분리 없음 교안 #34

이전 에이전트의 도구 호출 · 결과가 공유 messages로 다음 에이전트에 그대로

`agent_nodes.py:427`

2 기억 이중 주입 교안 #28

훅이 JSON으로 넣은 기억을 프롬프트가 도구로 다시 검색하라고 지시

`supervisor.py:394 · planning_prompt`

3 검색 도구 이름 불일치 교안 #46

설정은 x-amz-…-search(하이픈), 실제는 x_amz_…_search(밑줄) → 탈락

`agent_config.yaml:47`

4 계획 단계 미전달 교안 #13

집중 질문에 원래 질문만 있고, 계획의 단계 설명은 로그에만 쓰임

`agent_nodes.py:163`

5 필터 없는 50개 기억 교안 #29

인프라 지식 최대 50개를 점수 필터 없이 레코드 전체 JSON으로 삽입

`constants.py:190 · hooks.py:57`

6 지침 충돌 교안 #12

계획 지침 '최대 2–3단계' vs 응답 형식 'steps 3–5개'

`supervisor_planning_prompt.txt`

실습 과제: 몇 줄로 고치는 컨텍스트

교안 #13–46

1 다음 에이전트에는 결론만

함정 1

```
# agent_nodes.py:427
"messages": [AIMessage(
    content=f"[{self.name}]\n{agent_response}")],
# 도구 trace는 metadata에만 보관
```

2 기억은 한 경로로만

함정 2

```
# supervisor_planning_prompt.txt
# <memory_retrieval> … </memory_retrieval> 블록 삭제
# (훅이 이미 시스템 메시지에 넣음)
```

3 검색 도구 이름 고치기

함정 3

```
# config/agent_config.yaml
global_tools:
  - x_amz_bedrock_agentcore_search
```

4 계획 단계 전달하기

함정 4

```
# agent_nodes.py:163
step = state["metadata"].get("routing_reasoning", "")
agent_prompt += f"\nYour assigned step: {step}"
```

측정하기 고치기 전후로 같은 질문을 실행하고, 에이전트별 AIMessage.usage_metadata의 input_tokens 합계와 최종 보고 품질을 비교

정리

공유 상태에서는 분리를 직접 설계한다

기록 · 선택

기록은 네 갈래로 공들였지만, 선택은 계획 단계에만 있고 점수 필터 없이 두 번 들어간다

압축 · 분리

압축은 저장과 보고 쪽 절단뿐이고, 공유 messages가 에이전트 사이의 경계를 지운다

도구 설계

에이전트별 5개 바인딩은 교안 그대로 — 단, 설정 이름 하나가 검색 도구를 지웠다

A2A 샘플은 런타임 경계가 분리되었지만, LangGraph처럼 상태를 공유하는 구조에서는 무엇을 다음 노드에 넘길지 코드로 직접 정해야 합니다.